

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BCSE307P — Compiler Design Laboratory · Fall Semester 2026–27

PHASE 1 — REVIEW 1 REPORT
PROBLEM DEFINITION AND SYSTEM DESIGN

Tabula: A Spreadsheet Engine with a Compiled Formula Language, Dependency- Graph Analysis, and Incremental Recomputation

An individual Compiler Design laboratory project implementing a complete formula compiler (lexer, parser, semantic analyzer, optimizer, bytecode generator, and evaluation VM) together with whole-sheet static analysis: dependency extraction, circular-reference detection, and minimal recomputation.

Student Name	Shashank Bhat M
Registration No.	24BDS0390
Programme	B.Tech — Computer Science and Engineering
Course	BCSE307P Compiler Design Laboratory
Faculty Guide	Prof. Ranjithkumar
Date	September 2026

1. Abstract

Spreadsheets are the most widely used end-user programming environment in the world, yet the language-processing machinery that powers them is rarely examined: every cell formula must be tokenized, parsed, semantically checked, compiled, and — critically — re-evaluated efficiently and in the correct order whenever any cell it depends on changes. This project, **Tabula**, implements that machinery from scratch as a transparent, fully inspectable system. Each formula (e.g. `=SUM(A1:B4)*rate`) passes through a complete, hand-written compiler pipeline: lexical analysis, recursive-descent/Pratt parsing into an abstract syntax tree, semantic analysis against a function registry and a named-range symbol table, AST-level optimization (constant folding, algebraic simplification, branch elimination), and code generation to a custom stack-machine bytecode executed by a small evaluation VM with faithful spreadsheet error-value semantics (`#DIV/0!`, `#NAME?`, `#REF!`, `#VALUE!`, `#CYCLE!`). Above the per-formula pipeline, a sheet-level engine performs whole-program static analysis: it extracts every formula's cell dependencies during compilation, maintains a bidirectional dependency graph, detects circular references (reporting the actual cycle path), and performs **incremental recomputation** — after any edit, only the transitive dependents of the edited cell are re-evaluated, in topological order. An `explain` command exposes every intermediate artifact of any cell's compilation, making the system its own teaching tool. The engine is implemented in pure Python with no parser generators or expression libraries, and is validated by unit, scenario, and benchmark tests, including the invariant that incremental results always equal a full recomputation.

2. Problem Statement

A spreadsheet is, computationally, a network of thousands of tiny interdependent programs. Supporting it correctly requires solving a genuinely compiler-shaped problem with two layers:

1. **Per-formula:** each formula is source code in a small expression language and must be lexed, parsed, checked for semantic validity (unknown functions, wrong argument counts, invalid references), compiled, and evaluated — with errors contained so that one malformed cell never corrupts the rest of the sheet.
2. **Per-sheet:** the collection of formulas induces a dependency graph. The engine must detect circular references, order evaluation so every cell is computed after its inputs, and — because sheets are edited interactively — avoid the naïve strategy of recomputing all N cells on every keystroke, which scales as $O(N)$ per edit and makes large sheets unusable.

No compact, transparent, single-student implementation exists that demonstrates how these two layers work together. Commercial engines (Microsoft Excel, Google Sheets) solve the problem at scale but are closed and internally invisible; textbook compiler projects, conversely, compile one standalone program and never encounter incremental, graph-structured recomputation. **The problem addressed by this project is to design and build a complete, observable spreadsheet engine in which both the formula compiler and the dependency-driven incremental recomputation are implemented from first principles and can be inspected at every stage.**

3. Motivation

- **Compiler concepts with a visible payoff.** Every classical phase — scanning, parsing, symbol tables, semantic checking, IR, optimization, code generation, interpretation — is exercised by a system whose behavior a reviewer can verify instantly by editing a cell and watching exactly the right cells update.
- **Beyond the single-program compiler.** Spreadsheet recalculation is one of the few settings where classical compilation meets *incremental computation* and whole-program dependency analysis: cycle detection is a semantic check over the entire sheet, and topological scheduling decides evaluation order. These are concepts an ordinary course project never reaches.
- **Measurable optimization.** The project yields quantitative results required in later reviews: “editing one cell of a 10,000-formula sheet recomputed 14 cells, not 10,000”, and “constant folding removed k bytecode instructions and e dependency edges on the corpus”.
- **Familiarity of the domain.** Every evaluator knows Excel. The demonstration requires no explanation of *what* the system is — attention goes entirely to *how* it works.

4. Objectives

1. Design the Tabula formula language: token set, EBNF grammar, operator precedence, type-coercion rules, spreadsheet error-value semantics, and a library of ≈ 20 built-in functions.
2. Implement a hand-written **lexer** for formulas — cell references (`A1` , `$B$2`), ranges (`A1:B4`), numbers, strings, booleans, operators — with position-tracked lexical error reporting.
3. Implement a **recursive-descent parser** with Pratt precedence climbing producing an abstract syntax tree, reporting syntax errors with caret-annotated positions.
4. Implement **semantic analysis**: a function registry and named-range **symbol table**, unknown-name detection, arity and argument-kind checking, reference-bounds validation, and extraction of each formula’s precedent (dependency) set.
5. Maintain a sheet-level **dependency graph**; detect **circular references** and mark all cells of a cycle `#CYCLE!` with the cycle path reported.
6. Implement **optimization**: constant folding, algebraic simplification, and constant-condition branch elimination at the AST level, plus a bytecode peephole pass — each measurable and toggleable.
7. Compile formulas to **stack-machine bytecode** and implement the **evaluation VM** with error-value propagation and short-circuit `IF / AND / OR`.
8. Implement **incremental recomputation**: dirty-set closure over the dependency graph and re-evaluation in topological order, with per-edit statistics.
9. Provide an interactive REPL (`set` , `show` , `explain` , `deps` , `stats` , load/save) and a single-file HTML report of any sheet and any cell’s full compilation pipeline.
10. Validate the system with unit, scenario, and benchmark tests, including corpus-wide invariants (incremental $\equiv$ full recomputation; optimizer on/off value-equivalence).

5. Scope

5.1 In Scope

Area	Contents
Formula language	Number/string/boolean literals; relative and absolute cell references; ranges; named ranges; operators <code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> <code>^</code> (arithmetic), <code>&</code> (concatenation), <code>=</code> <code><></code> <code><</code> <code><=</code> <code>></code> <code>>=</code> (comparison), unary minus; function calls; parentheses.
Function library	<code>SUM</code> <code>AVG</code> <code>MIN</code> <code>MAX</code> <code>COUNT</code> <code>COUNTIF</code> <code>ROUND</code> <code>ABS</code> <code>SQRT</code> <code>MOD</code> <code>POWER</code> <code>IF</code> <code>AND</code> <code>OR</code> <code>NOT</code> <code>LEN</code> <code>CONCAT</code> <code>UPPER</code> <code>LOWER</code> <code>TRIM</code> <code>ISNUMBER</code> <code>ISBLANK</code>
Compiler	Lexer; parser + AST; semantic analyzer + symbol tables; AST optimizer; bytecode generator + peephole.
Engine	Sparse grid ($A-ZZ \times 1-10\,000$); dependency graph; cycle detection; full and incremental recomputation; error-value propagation.
Runtime	Stack VM; all five error values; runtime error localization.
Tooling	REPL/CLI; per-cell <code>explain</code> ; dependency queries; recompute statistics; TSV load/save; HTML report.
Quality	pytest suite; example and benchmark sheet corpus; user instructions.

5.2 Out of Scope (future enhancements)

Graphical UI (the CLI grid and HTML report are the interfaces); volatile functions (`NOW()`, `RAND()`) and date/time types; cross-sheet references; row/column insertion with reference rewriting; array/spill formulas and user-defined functions; charting, formatting, undo/redo, and concurrent editing.

6. Background Study

6.1 Compiler pipeline for expression languages

The classical phase structure — lexical analysis, syntax analysis, semantic analysis, intermediate representation, optimization, code generation, and execution (Aho et al. [1]) — applies directly to formula languages, with the simplification that a formula is a single expression rather than a statement sequence. For parsing, **Pratt’s top-down operator-precedence method** [2] is the standard technique for expression grammars with many precedence levels and mixed associativity (Tabula has seven tiers, with right-associative `^`); it integrates naturally into a hand-written recursive-descent parser and is well documented in modern treatments such as Nystrom [3].

6.2 Spreadsheet implementation technology

Sestoft [4] gives the authoritative account of spreadsheet internals: formulas compiled per cell, a **support/dependency graph** maintained between cells, evaluation ordered by that graph, and recalculation

performed either fully or minimally. Real engines follow the same shape — Excel maintains dependency chains and a “dirty” marking scheme so that recalculation touches only cells downstream of an edit [5]. Circular references are detected as cycles in the dependency graph, and error conditions are represented as first-class *error values* (`#DIV/0!`, `#NAME?`, ...) that propagate through arithmetic rather than aborting evaluation — a semantics Tabula reproduces faithfully.

6.3 Graph algorithms

Cycle detection uses depth-first search with the standard three-color scheme (a back edge to a “gray” vertex proves a cycle) [6]; evaluation order uses **Kahn’s topological-sorting algorithm** [7] restricted to the dirty subgraph. Both are implemented iteratively so that dependency chains thousands of cells deep cannot exhaust Python’s recursion limit.

6.4 Existing systems compared

System	What it demonstrates	Gap this project fills
Excel / Google Sheets	Industrial dependency-driven recalculation	Closed source; machinery invisible
Spreadsheet libraries (e.g. formula parsers)	Parsing only	No semantic analysis, optimization, IR, or incremental engine
Course compiler projects	Full pipeline on one program	No multi-program dependency analysis or incrementality
Tabula	Full pipeline per formula <i>plus</i> whole-sheet analysis, all inspectable	—

7. Compiler Design Concepts Involved

Course concept	Where it appears in Tabula
Lexical analysis, token classification	Hand-written formula scanner: references, ranges, literals, operators, maximal munch
Context-free grammars (EBNF)	Formula grammar (§10.2), seven precedence tiers
Top-down parsing, operator precedence	Recursive descent with Pratt precedence climbing
Error detection and recovery	Positioned lexical/syntax diagnostics with caret excerpts; per-cell containment — a bad formula becomes an error value, never corrupts the sheet
Abstract syntax trees	Formula AST; visitor-based checking, optimization, and code generation passes
Symbol table management	Function registry (arity, parameter kinds, purity); named-range table with declare/resolve/redefine

Semantic analysis	Name resolution, arity/kind checking, static type checks, reference-bounds validation; cycle detection as a whole-program semantic check
Dependency / dataflow analysis	Precedent extraction per formula; bidirectional dependency graph; transitive dirty-set closure
Intermediate representation	Stack-machine bytecode per formula with constant and reference pools (explicit AST $\rightarrow$ IR lowering)
Code optimization	Constant folding, algebraic simplification, branch elimination (AST); peephole (bytecode); incremental recomputation as computation-level optimization
Target code generation	Post-order AST walk emitting bytecode; short-circuit IF/AND/OR compiled to jumps
Interpretation and runtime environments	Stack VM over the grid as its store; error-value runtime semantics
Scheduling / topological ordering	Kahn's algorithm over the dirty subgraph for recomputation order

8. Proposed Methodology

The system is built as **two cooperating layers**, developed and tested bottom-up across the three course phases.

8.1 Micro layer — the per-formula compiler

Run whenever a cell's formula is set: *lexer* $\rightarrow$ *parser* $\rightarrow$ *semantic analyzer* $\rightarrow$ *AST optimizer* $\rightarrow$ *bytecode generator* $\rightarrow$ *chunk*. The semantic pass simultaneously extracts the formula's **precedent set** (all cells and ranges it reads), which is the interface to the macro layer. Optimization runs before final precedent extraction, so a folded branch such as `=IF(FALSE, B1, 7)` ends with *no* dependency on `B1` — the optimizer visibly changes the dependency graph, not merely execution speed.

8.2 Macro layer — the sheet engine

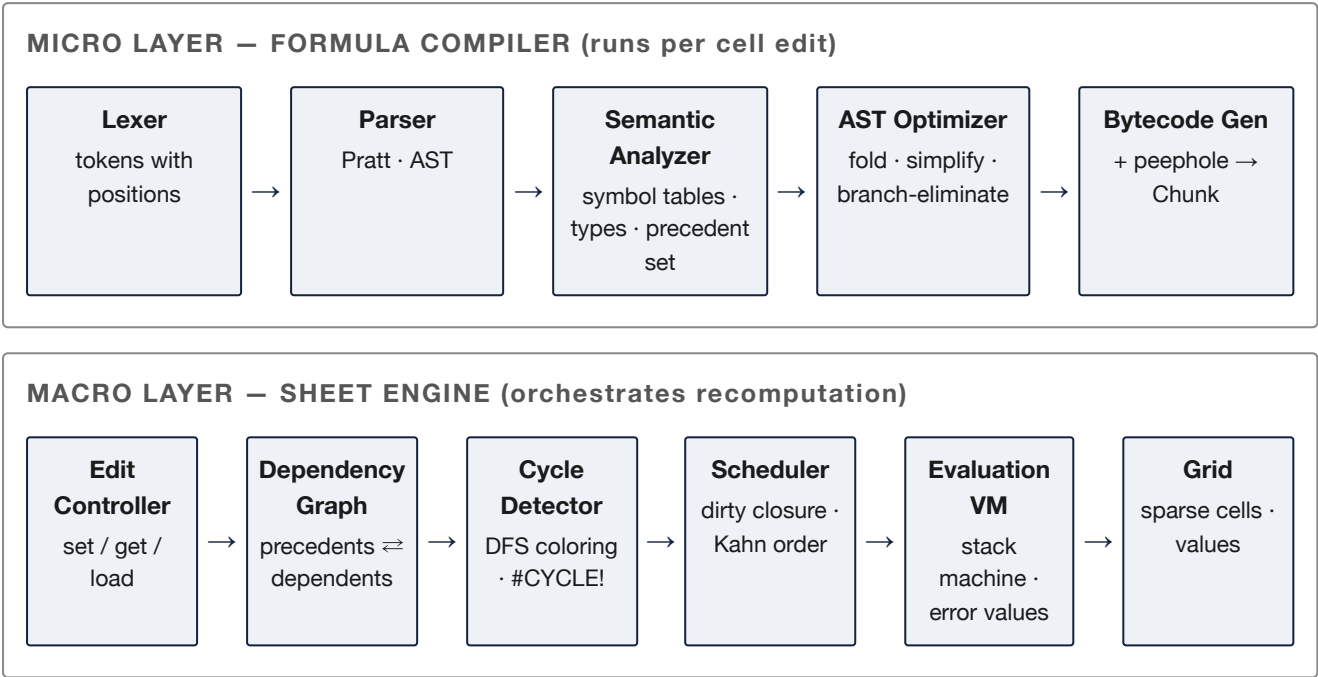
On every edit: (1) compile the formula through the micro layer; (2) update the dependency graph edges by diffing old and new precedents; (3) run cycle detection from the edited cell, marking any cycle's members `#CYCLE!` with the path reported; (4) compute the dirty set as the transitive dependents of the edited cell; (5) re-evaluate only the dirty set, in Kahn topological order; (6) record statistics (cells recomputed vs. total, VM instructions, elapsed time). A full recomputation is the same code path with the dirty set equal to all formula cells — one algorithm, exercised two ways.

8.3 Correctness discipline

Two corpus-wide invariants are enforced by tests from the first phase onward: (i) after any edit script, the incrementally maintained grid equals a from-scratch evaluation of the entire sheet; (ii) enabling or disabling the optimizer never changes any cell value. Error handling follows real-spreadsheet semantics:

errors are typed values that propagate through dependent formulas; the engine process itself never crashes on any input.

9. System Architecture



Every micro-layer artifact tees into the `explain`/HTML visualizer; every stage reports positioned diagnostics into a shared diagnostics module.

9.1 Module decomposition

tabula/	
└ cli.py	REPL: commands, grid rendering
└ diagnostics.py	positioned errors, caret excerpts
└ refs.py	CellRef/RangeRef parse · format · bounds
└ values.py	value model: Number/Text/Boolean/Blank/Error
└ lexer/	token.py, lexer.py
└ parser/	ast.py, parser.py
└ sema/	functions.py (registry), names.py (named ranges), checker.py
└ opt/	passes.py (fold / simplify / branch-eliminate)
└ codegen/	bytecode.py (OpCodes, Chunk, peephole)
└ vm/	vm.py (stack evaluation)
└ engine/	cell.py · grid.py (TSV load/save) · graph.py (edges, cycles, closure, Kahn) · engine.py (edit controller, stats)
└ viz/	explain.py (per-cell dumps) · report.py (HTML report)
└ tests/	unit · scenario · benchmark
└ examples/	budget.tsv, grades.tsv, ...

Dependency rule: micro-layer packages depend only on earlier stages plus shared value/reference/diagnostic modules; the engine consumes the micro layer as a black box (`compile_formula(text) → Chunk | diagnostics`); the visualizer depends on everything and nothing depends on it.

10. Language Specification (summary)

10.1 Tokens

Class	Pattern / members
Cell reference	<code>[\$]?[A-Z]{1,2}[\$]?[0-9]+</code> — <code>A1</code> , <code>\$B\$2</code> , <code>AA10</code> ; bounds <code>A-ZZ</code> × <code>1-10 000</code>
Range	<code>ref ':' ref</code> — combined by the parser
Number / String / Boolean	<code>12</code> , <code>3.5</code> , <code>.5</code> ; <code>"text"</code> with <code>""</code> escape; <code>TRUE</code> / <code>FALSE</code>
Identifier	function name or named range, case-insensitive
Operators / delimiters	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> <code>^</code> <code>&</code> <code>=</code> <code><></code> <code><</code> <code><=</code> <code>></code> <code>>=</code> <code>(</code> <code>)</code> <code>,</code> <code>:</code>

10.2 Grammar (EBNF)

```
formula      = "=" expression EOF ;
expression   = comparison ;
comparison   = concat { ( "=" | "<>" | "<" | "<=" | ">" | ">=" ) concat } ;
concat       = additive { "&" additive } ;
additive     = multiplicative { ( "+" | "-" ) multiplicative } ;
multiplicative = power { ( "*" | "/" | "%" ) power } ;
power        = unary [ "^" power ] ;                      (* right-associative *)
unary        = "-" unary | primary ;
primary      = NUMBER | STRING | BOOLEAN | range | CELLREF | IDENT
              | IDENT "(" [ args ] ")" | "(" expression ")" ;
range        = CELLREF ":" CELLREF ;
args         = expression { "," expression } ;
```

10.3 Semantics highlights

- **Types:** Number, Text, Boolean, Blank, Error; Range only as an argument to range-accepting functions (`=A1:B2 + 1` is a semantic error).
- **Coercions:** Blank $\rightarrow$ 0/"" by context; Boolean $\rightarrow$ 1/0 numerically; Text never implicitly numeric (`= "2" + 1  $\Rightarrow$  #VALUE!`); `&` coerces to Text.
- **Error values:** `#NAME?` unknown function/name · `#REF!` out-of-bounds reference · `#VALUE!` type error · `#DIV/0!` division/modulo by zero · `#CYCLE!` circular reference (path reported). Any error operand makes the result that error; `IF` evaluates only the taken branch; `COUNT`, `ISNUMBER`, `ISBLANK` follow their per-function policies.

11. Technology Stack

Category	Choice	Justification
----------	--------	---------------

Implementation language	Python 3.11+	Recommended by the course manual; concise for tree- and graph-heavy code
Parser construction	Hand-written (no Lex/Yacc/Bison/ANTLR)	Every algorithm individually implemented and explainable in viva; full control of error messages
Expression evaluation	Own bytecode VM (Python <code>eval</code> prohibited by design)	Genuine code-generation and interpretation phases
Testing	pytest	Unit, scenario (edit-script), and benchmark tests
Version control	Git + GitHub (private)	Required by manual; commit history as development log
Editor / OS	VS Code · macOS/Linux/Windows	Portable, standard-library-only runtime

12. Initial Implementation (Prototype)

The Phase 1 prototype is **implemented and working**. It realizes the front of the micro layer together with a minimal grid and REPL, proving the language design end-to-end before the full engine is built. The code is pure Python 3.11 with no third-party runtime dependencies, organized exactly as the architecture of §9.1 prescribes, so Phase 2 and 3 modules slot in without restructuring.

12.1 Implemented modules

Module	Lines	Contents
<code>tabula/refs.py</code>	70	Cell/range references: <code>A1</code> , <code>\$B\$2</code> , <code>AA10</code> parsing and formatting, column↔index conversion, bounds enforcement ($A-ZZ \times 1-10\,000$), range cell enumeration
<code>tabula/values.py</code>	66	Value model (Number, Text, Boolean, Blank) and the five error values; coercion rules (<code>to_number</code> , <code>to_text</code>) and display formatting
<code>tabula/lexer.py</code>	119	Hand-written scanner: all token classes, maximal munch for <code><=</code> <code>>=</code> <code><></code> , doubled-quote string escapes, reference-vs-identifier classification, positioned lexical errors
<code>tabula/parser.py</code>	197	Pratt precedence-climbing parser over the §10.2 grammar, AST node definitions, and the box-drawing AST printer used by <code>explain</code>
<code>tabula/evaluator.py</code>	157	Direct AST evaluation over the grid with error propagation, lazy <code>IF</code> branches, range flattening, and active-cell cycle detection (<i>interim</i> — replaced by the bytecode VM in Phase 3)
<code>tabula/engine.py</code>	108	Grid, edit controller (<code>set</code> / <code>value</code>), caret diagnostics, grid renderer, and <code>explain</code>
<code>tabula/cli.py</code>	77	REPL with interactive and batch (script) modes

tests/	220	49 tests: lexer (token classes, escapes, maximal munch, errors), parser (precedence/associativity table, error positions), evaluator (semantics, coercions, all error values, cycles, containment)
--------	-----	--

12.2 Representative code — the Pratt parsing loop

Binding powers encode the seven precedence tiers of §10.2; the right-associativity flag is what makes `=2^3^2` evaluate as $2^{(3^2)} = 512$ rather than $(2^3)^2 = 64$.

```
# tabula/parser.py – binding powers: (left_bp, right_associative)
_BINARY_BP = {
    TokenType.EQ: (10, False), TokenType.NE: (10, False),
    TokenType.LT: (10, False), TokenType.LE: (10, False),
    TokenType.GT: (10, False), TokenType.GE: (10, False),
    TokenType.AMP: (20, False),                               # & concatenation
    TokenType.PLUS: (30, False), TokenType.MINUS: (30, False),
    TokenType.STAR: (40, False), TokenType.SLASH: (40, False),
    TokenType.PERCENT: (40, False),
    TokenType.CARET: (50, True),                               # ^ right-associative
}

def parse_expression(self, min_bp: int = 0):
    left = self.parse_unary()
    while True:
        tok = self.peek()
        bp = _BINARY_BP.get(tok.type)
        if bp is None or bp[0] < min_bp:
            return left
        self.advance()
        lbp, right_assoc = bp
        next_min = lbp if right_assoc else lbp + 1      # hold bp if right-assoc
        right = self.parse_expression(next_min)
        left = Binary(tok.lexeme, left, right)
```

12.3 Captured demonstration session

Verbatim output of `python3 -m tabula.cli examples/demo.txt` — the Review 1 demonstration script:

```

tabula> set A1 10
tabula> set A2 25
tabula> set A3 40
tabula> set B1 =SUM(A1:A3)*2
tabula> set B2 =AVG(A1:A3)
tabula> set B3 =IF(B2>20, "high", "low")
tabula> show
      A   B
1    10  150
2    25   25
3    40  high
tabula> set A2 35                                ← edit an input
tabula> get B1
B1 = 170    (=SUM(A1:A3)*2)                       ← dependent formula updated
tabula> explain B1
cell   : B1
raw    : =SUM(A1:A3)*2
kind   : formula
tokens : SUM ( A1 : A3 ) * 2
types  : IDENT LPAREN CELLREF COLON CELLREF RPAREN STAR NUMBER
ast    :
  Binary(*)
  ├── Call(SUM)
  │   └── Range(A1:A3)
  └── Number(2)
value  : 170
tabula> set C1 =SUM(A1:A3                          ← deliberate syntax error
error[SyntaxError] in C1: expected ')' after argument list
    =SUM(A1:A3
      ^

tabula> set C2 =10/0
tabula> set A5 =B5
tabula> set B5 =A5                                ← deliberate circular reference
tabula> show
      A       B       C
1    10      170     #VALUE!
2    35      28.33333333 #DIV/0!
3    40      high
5    #CYCLE!  #CYCLE!

```

The final grid demonstrates the error model of §10.3 in one view: the broken formula in **C1** yields **#VALUE!**, division by zero in **C2** yields **#DIV/0!**, the mutually referencing pair **A5 / B5** is detected as a cycle, and every other cell continues to evaluate correctly — per-cell error containment working as designed.

12.4 Test results

```

$ python3 -m pytest tests/ -q
..... [100%]
49 passed in 0.03s

```

Test module	Tests	Coverage
-------------	-------	----------

<code>test_lexer.py</code>	15	Every token class; number forms (<code>12</code> , <code>3.5</code> , <code>.5</code>); doubled-quote escapes; absolute-reference anchors; maximal munch (<code><=</code> vs <code><</code>); reference-vs-identifier classification incl. the bound <code>ZZ10000</code> / <code>ZZ10001</code> ; token positions; three lexical-error kinds
<code>test_parser.py</code>	14	Table-driven precedence and associativity (8 cases incl. right-associative <code>^</code> and unary binding); ranges, calls, nested calls; four syntax errors with asserted message <i>and</i> position
<code>test_evaluator.py</code>	20	Expression semantics and coercions; aggregates over ranges; edit propagation; blank-cell handling; lazy <code>IF</code> (proved by an untaken <code>1/0</code> branch); four error values; error propagation; cycle detection; per-cell containment; caret diagnostics

12.5 Status against the phase plan

Phase 1 tasks T1.1–T1.10 of the project task plan are complete: repository setup and module skeleton, specification freeze, the reference/value/diagnostic infrastructure, the lexer, the Pratt parser, the prototype evaluator with REPL, `explain` v0, and this review pack. The language specification (§10) is frozen as of this review.

Phase 2 proceeds to the function registry and named-range symbol tables, full semantic analysis, precedent extraction, the dependency graph with cycle detection, and graph-ordered recomputation; Phase 3 delivers the AST optimizer, the bytecode compiler and VM (replacing the interim evaluator), true incremental recomputation with benchmark evidence, and the complete visualizer and HTML report.

13. Expected Outcomes

1. A working spreadsheet engine evaluating the full function library with faithful error-value semantics.
2. Live circular-reference detection with the cycle path reported, and per-cell error containment.
3. Measured evidence of incrementality (cells recomputed per edit vs. sheet size on 10,000-cell benchmarks) and of optimization (bytecode reduction, dependency edges removed by folding).
4. A per-cell `explain` pipeline X-ray and a self-contained HTML report of the whole sheet.
5. Complete documentation and tests, with all algorithms individually explainable for the vivas.

14. References

1. A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Pearson, 2006.
2. V. R. Pratt, “Top Down Operator Precedence,” *Proceedings of the 1st ACM Symposium on Principles of Programming Languages (POPL)*, 1973.
3. R. Nystrom, *Crafting Interpreters*, Genever Benning, 2021. (Pratt parsing and bytecode virtual machines.)
4. P. Sestoft, *Spreadsheet Implementation Technology: Basics and Extensions*, MIT Press, 2014.
5. Microsoft Learn, “Excel Recalculation,” Office Client Development documentation.

6. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
(Depth-first search, cycle detection, topological sort.)
7. A. B. Kahn, "Topological Sorting of Large Networks," *Communications of the ACM*, 5(11), 1962.

Appendix A — Complete Source Code

Listings are the verbatim contents of the Phase 1 repository at the time of this review (Python 3.11, standard library only). Line numbers are shown for reference during the viva.

A.1 `tabula/refs.py` — Cell and range references, bounds, column conversion (70 lines)

```
1  """Cell and range references: parsing, formatting, bounds (A-ZZ x 1-10000)."""
2  from __future__ import annotations
3
4  import re
5  from dataclasses import dataclass
6
7  MAX_ROW = 10_000
8  MAX_COL = 26 + 26 * 26 # A..Z, AA..ZZ
9
10 _REF_RE = re.compile(r"^(\\$?)([A-Z]{1,2})(\\$?)([0-9]+)$")
11
12
13 def col_to_index(letters: str) -> int:
14     """'A' -> 1, 'Z' -> 26, 'AA' -> 27, 'ZZ' -> 702."""
15     n = 0
16     for ch in letters:
17         n = n * 26 + (ord(ch) - ord("A") + 1)
18     return n
19
20
21 def index_to_col(n: int) -> str:
22     letters = ""
23     while n > 0:
24         n, rem = divmod(n - 1, 26)
25         letters = chr(ord("A") + rem) + letters
26     return letters
27
28
29 @dataclass(frozen=True)
30 class CellRef:
31     col: int # 1-based
32     row: int # 1-based
33     abs_col: bool = False
34     abs_row: bool = False
35
36     @staticmethod
37     def parse(text: str) -> "CellRef | None":
38         m = _REF_RE.match(text.upper())
39         if not m:
40             return None
41         dc, letters, dr, digits = m.groups()
42         col, row = col_to_index(letters), int(digits)
43         if row < 1 or row > MAX_ROW or col > MAX_COL:
44             return None
45         return CellRef(col, row, abs_col=dc == "$", abs_row=dr == "$")
46
47     def key(self) -> tuple[int, int]:
48         return (self.col, self.row)
49
50     def __str__(self) -> str:
51         return (
52             (" $" if self.abs_col else "") + index_to_col(self.col)
53             + (" $" if self.abs_row else "") + str(self.row)
54         )
55
56
57 @dataclass(frozen=True)
58 class RangeRef:
59     start: CellRef
```

```

60     end: CellRef
61
62     def cells(self):
63         c1, c2 = sorted((self.start.col, self.end.col))
64         r1, r2 = sorted((self.start.row, self.end.row))
65         for col in range(c1, c2 + 1):
66             for row in range(r1, r2 + 1):
67                 yield CellRef(col, row)
68
69     def __str__(self) -> str:
70         return f"{self.start}:{self.end}"

```

A.2 `tabula/values.py` — Value model, error values, coercion and display rules (66 lines)

```

1  """Value model: numbers, text, booleans, blanks, and spreadsheet error values."""
2  from __future__ import annotations
3
4  from dataclasses import dataclass
5
6
7  @dataclass(frozen=True)
8  class ErrorValue:
9      code: str # '#DIV/0!', '#VALUE!', '#REF!', '#NAME?', '#CYCLE!'
10     detail: str = ""
11
12     def __str__(self) -> str:
13         return self.code
14
15
16     DIV0 = ErrorValue("#DIV/0!")
17     VALUE_ERR = ErrorValue("#VALUE!")
18     REF_ERR = ErrorValue("#REF!")
19     NAME_ERR = ErrorValue("#NAME?")
20     CYCLE_ERR = ErrorValue("#CYCLE!")
21
22     BLANK = None # an empty cell reads as Blank
23
24
25     def is_error(v) -> bool:
26         return isinstance(v, ErrorValue)
27
28
29     def to_number(v):
30         """Coerce a value to a number per Tabula rules; ErrorValue on failure.
31
32         Blank -> 0, Boolean -> 1/0, Text is never implicitly numeric.
33         """
34         if is_error(v):
35             return v
36         if v is BLANK:
37             return 0
38         if isinstance(v, bool):
39             return 1 if v else 0
40         if isinstance(v, (int, float)):
41             return v
42         return VALUE_ERR
43
44
45     def to_text(v) -> str | ErrorValue:
46         if is_error(v):
47             return v
48         if v is BLANK:
49             return ""
50         if isinstance(v, bool):
51             return "TRUE" if v else "FALSE"
52         if isinstance(v, float):
53             if v == int(v):
54                 return str(int(v))
55             # Spreadsheet-style display: significant digits, no float artefacts.
56             return f"{v:.10g}"
57         return str(v)

```

```

58
59
60 def display(v) -> str:
61     """Human-readable form for the grid renderer."""
62     if v is BLANK:
63         return ""
64     if is_error(v):
65         return v.code
66     return to_text(v)

```

A.3 `tabula/lexer.py` — Hand-written formula scanner (119 lines)

```

1  """Hand-written scanner for Tabula formulas.
2
3  Tokenizes the text after the leading '='. Every token carries its start
4  column within the formula text so diagnostics can point at the exact spot.
5  """
6  from __future__ import annotations
7
8  from dataclasses import dataclass
9  from enum import Enum, auto
10
11  from .refs import CellRef
12
13
14  class TokenType(Enum):
15      NUMBER = auto()
16      STRING = auto()
17      BOOL = auto()
18      CELLREF = auto()
19      IDENT = auto()
20      PLUS = auto(); MINUS = auto(); STAR = auto(); SLASH = auto()
21      PERCENT = auto(); CARET = auto(); AMP = auto()
22      EQ = auto(); NE = auto(); LT = auto(); LE = auto(); GT = auto(); GE = auto()
23      LPAREN = auto(); RPAREN = auto(); COMMA = auto(); COLON = auto()
24      EOF = auto()
25
26
27  @dataclass
28  class Token:
29      type: TokenType
30      lexeme: str
31      value: object # parsed literal / CellRef where applicable
32      pos: int      # 0-based start column in the formula text
33
34
35  class LexError(Exception):
36      def __init__(self, message: str, pos: int):
37          super().__init__(message)
38          self.message, self.pos = message, pos
39
40
41  _SINGLE = {
42      "+": TokenType.PLUS, "-": TokenType.MINUS, "*": TokenType.STAR,
43      "/": TokenType.SLASH, "%": TokenType.PERCENT, "^": TokenType.CARET,
44      "&": TokenType.AMP, "=": TokenType.EQ, "(": TokenType.LPAREN,
45      ")": TokenType.RPAREN, ",": TokenType.COMMA, ":": TokenType.COLON,
46  }
47
48
49  def tokenize(text: str) -> list[Token]:
50      toks: list[Token] = []
51      i, n = 0, len(text)
52      while i < n:
53          ch = text[i]
54          if ch.isspace():
55              i += 1
56              continue
57          # -- numbers: 12, 3.5, .5 (maximal munch)
58          if ch.isdigit() or (ch == "." and i + 1 < n and text[i + 1].isdigit()):
59              start = i

```

```

60         while i < n and text[i].isdigit():
61             i += 1
62         if i < n and text[i] == ".":
63             i += 1
64             while i < n and text[i].isdigit():
65                 i += 1
66             lexeme = text[start:i]
67             toks.append(Token(TokType.NUMBER, lexeme, float(lexeme), start))
68             continue
69         # -- strings: "..." with "" as the escaped quote
70         if ch == '"':
71             start, i, parts = i, i + 1, []
72             while True:
73                 if i >= n:
74                     raise LexError("unterminated string literal", start)
75                 if text[i] == '"':
76                     if i + 1 < n and text[i + 1] == '"': # escaped quote
77                         parts.append('"')
78                         i += 2
79                         continue
80                 i += 1
81                 break
82             parts.append(text[i])
83             i += 1
84             toks.append(Token(TokType.STRING, text[start:i], "".join(parts), start))
85             continue
86         # -- references, booleans, identifiers
87         if ch.isalpha() or ch in "_$":
88             start = i
89             while i < n and (text[i].isalnum() or text[i] in "_$."):
90                 i += 1
91             word = text[start:i]
92             ref = CellRef.parse(word)
93             upper = word.upper()
94             if ref is not None:
95                 toks.append(Token(TokType.CELLREF, word, ref, start))
96             elif upper in ("TRUE", "FALSE"):
97                 toks.append(Token(TokType.BOOL, word, upper == "TRUE", start))
98             elif "$" in word:
99                 raise LexError(f"malformed cell reference '{word}'", start)
100             else:
101                 toks.append(Token(TokType.IDENT, word, upper, start))
102             continue
103         # -- two-char operators before one-char (maximal munch)
104         two = text[i:i + 2]
105         if two == "<>":
106             toks.append(Token(TokType.NE, two, None, i)); i += 2; continue
107         if two == "<=":
108             toks.append(Token(TokType.LE, two, None, i)); i += 2; continue
109         if two == ">=":
110             toks.append(Token(TokType.GE, two, None, i)); i += 2; continue
111         if ch == "<":
112             toks.append(Token(TokType.LT, ch, None, i)); i += 1; continue
113         if ch == ">":
114             toks.append(Token(TokType.GT, ch, None, i)); i += 1; continue
115         if ch in _SINGLE:
116             toks.append(Token(_SINGLE[ch], ch, None, i)); i += 1; continue
117         raise LexError(f"illegal character {ch!r}", i)
118     toks.append(Token(TokType.EOF, "", None, n))
119     return toks

```

A.4 `tabula/parser.py` — Pratt parser, AST definitions, AST printer (197 lines)

```

1  """Recursive-descent parser with Pratt precedence climbing for expressions.
2
3  Grammar (design.md section 4): seven precedence tiers, comparison lowest,
4  '^' right-associative, unary minus binding tighter than '^' operand position.
5  """
6  from __future__ import annotations
7
8  from dataclasses import dataclass, field

```

```

9
10 from .lexer import Token, TokenType, tokenize, LexError
11 from .refs import CellRef, RangeRef
12
13
14 # ----- AST -----
15 @dataclass
16 class Number:
17     value: float
18
19
20 @dataclass
21 class Text:
22     value: str
23
24
25 @dataclass
26 class Bool:
27     value: bool
28
29
30 @dataclass
31 class Ref:
32     ref: CellRef
33
34
35 @dataclass
36 class RangeNode:
37     rng: RangeRef
38
39
40 @dataclass
41 class Name:
42     ident: str
43
44
45 @dataclass
46 class Unary:
47     op: str
48     operand: object
49
50
51 @dataclass
52 class Binary:
53     op: str
54     left: object
55     right: object
56
57
58 @dataclass
59 class Call:
60     fname: str
61     args: list = field(default_factory=list)
62
63
64 class ParseError(Exception):
65     def __init__(self, message: str, pos: int):
66         super().__init__(message)
67         self.message, self.pos = message, pos
68
69
70 # binding powers: (left_bp, right_associative)
71 _BINARY_BP = {
72     TokenType.EQ: (10, False), TokenType.NE: (10, False),
73     TokenType.LT: (10, False), TokenType.LE: (10, False),
74     TokenType.GT: (10, False), TokenType.GE: (10, False),
75     TokenType.AMP: (20, False),
76     TokenType.PLUS: (30, False), TokenType.MINUS: (30, False),
77     TokenType.STAR: (40, False), TokenType.SLASH: (40, False),
78     TokenType.PERCENT: (40, False),
79     TokenType.CARET: (50, True),
80 }
81
82

```

```

83 class Parser:
84     def __init__(self, tokens: list[Token]):
85         self.toks = tokens
86         self.i = 0
87
88     def peek(self) -> Token:
89         return self.toks[self.i]
90
91     def advance(self) -> Token:
92         tok = self.toks[self.i]
93         self.i += 1
94         return tok
95
96     def expect(self, ttype: TokType, what: str) -> Token:
97         if self.peek().type is ttype:
98             return self.advance()
99         raise ParseError(f"expected {what}", self.peek().pos)
100
101     # -- Pratt loop -----
102     def parse_expression(self, min_bp: int = 0):
103         left = self.parse_unary()
104         while True:
105             tok = self.peek()
106             bp = _BINARY_BP.get(tok.type)
107             if bp is None or bp[0] < min_bp:
108                 return left
109             self.advance()
110             lbp, right_assoc = bp
111             next_min = lbp if right_assoc else lbp + 1
112             right = self.parse_expression(next_min)
113             left = Binary(tok.lexeme, left, right)
114
115     def parse_unary(self):
116         if self.peek().type is TokType.MINUS:
117             tok = self.advance()
118             return Unary("-", self.parse_unary())
119         return self.parse_primary()
120
121     def parse_primary(self):
122         tok = self.advance()
123         if tok.type is TokType.NUMBER:
124             return Number(tok.value)
125         if tok.type is TokType.STRING:
126             return Text(tok.value)
127         if tok.type is TokType.BOOL:
128             return Bool(tok.value)
129         if tok.type is TokType.CELLREF:
130             if self.peek().type is TokType.COLON: # range A1:B4
131                 self.advance()
132                 end = self.expect(TokType.CELLREF, "cell reference after ':'")
133                 return RangeNode(RangeRef(tok.value, end.value))
134             return Ref(tok.value)
135         if tok.type is TokType.IDENT:
136             if self.peek().type is TokType.LPAREN: # function call
137                 self.advance()
138                 args = []
139                 if self.peek().type is not TokType.RPAREN:
140                     args.append(self.parse_expression())
141                     while self.peek().type is TokType.COMMA:
142                         self.advance()
143                         args.append(self.parse_expression())
144                     self.expect(TokType.RPAREN, "'')' after argument list")
145                 return Call(tok.value, args)
146             return Name(tok.value)
147         if tok.type is TokType.LPAREN:
148             inner = self.parse_expression()
149             self.expect(TokType.RPAREN, "'')'")
150             return inner
151         what = f"unexpected {tok.lexeme!r}" if tok.lexeme else "unexpected end of formula"
152         raise ParseError(what, tok.pos)
153
154     def parse_formula(text: str):
155         """Parse formula body (text after '='). Raises LexError/ParseError."""
156

```

```

157     tokens = tokenize(text)
158     parser = Parser(tokens)
159     ast = parser.parse_expression()
160     trailing = parser.peek()
161     if trailing.type is not TokType.EOF:
162         raise ParseError(f"unexpected {trailing.lexeme!r} after expression", trailing.pos)
163     return ast
164
165
166 # ----- AST pretty-printer (explain v0) -----
167 def dump_ast(node, prefix: str = "", is_last: bool = True,
168             is_root: bool = True) -> list[str]:
169     label = {
170         Number: lambda n: f"Number({values_fmt(n.value)}",
171         Text: lambda n: f'Text("{n.value}")',
172         Bool: lambda n: f"Bool({n.value})",
173         Ref: lambda n: f"Ref({n.ref})",
174         RangeNode: lambda n: f"Range({n.rng})",
175         Name: lambda n: f"Name({n.ident})",
176         Unary: lambda n: f"Unary({n.op})",
177         Binary: lambda n: f"Binary({n.op})",
178         Call: lambda n: f"Call({n.fname})",
179     }[type(node)](node)
180     connector = "" if is_root else ("└─ " if is_last else "┌─ ")
181     lines = [prefix + connector + label]
182     children = []
183     if isinstance(node, Unary):
184         children = [node.operand]
185     elif isinstance(node, Binary):
186         children = [node.left, node.right]
187     elif isinstance(node, Call):
188         children = node.args
189     child_prefix = prefix + ("    " if is_root else ("    " if is_last else "|  "))
190     for k, child in enumerate(children):
191         last = k == len(children) - 1
192         lines.extend(dump_ast(child, child_prefix, last, is_root=False))
193     return lines
194
195
196 def values_fmt(v: float) -> str:
197     return str(int(v)) if v == int(v) else str(v)

```

A.5 `tabula/evaluator.py` — Direct AST evaluation (interim — replaced by the VM in Phase 3) (157 lines)

```

1  """Phase 1 prototype: direct AST evaluation over the grid.
2
3  Interim component — replaced by the bytecode VM in Phase 3. On-demand
4  recursive evaluation with an active-cell set for dynamic cycle detection
5  (Phase 2 replaces this with the dependency graph + topological order).
6  """
7  from __future__ import annotations
8
9  from . import parser as P
10 from .refs import CellRef
11 from .values import (BLANK, CYCLE_ERR, DIV0, NAME_ERR, VALUE_ERR,
12                     ErrorValue, is_error, to_number, to_text)
13
14
15 def _num_args(values):
16     """Coerce operands for arithmetic; first error wins."""
17     out = []
18     for v in values:
19         n = to_number(v)
20         if is_error(n):
21             return n
22         out.append(n)
23     return out
24
25
26 class Evaluator:
27     def __init__(self, grid):

```

```

28         self.grid = grid          # provides raw_cell(ref) -> (kind, ast_or_value)
29         self.active: set = set()   # cells currently being evaluated (cycle check)
30
31     # -- cell protocol -----
32     def value_of(self, ref: CellRef):
33         key = ref.key()
34         if key in self.active:
35             return CYCLE_ERR
36         cell = self.grid.cells.get(key)
37         if cell is None:
38             return BLANK
39         if cell.kind == "literal":
40             return cell.payload
41         if cell.kind == "broken":      # formula that failed to compile
42             return VALUE_ERR
43         self.active.add(key)
44         try:
45             return self.eval(cell.payload)
46         finally:
47             self.active.discard(key)
48
49     # -- expression evaluation -----
50     def eval(self, node):
51         method = getattr(self, f"_eval_{type(node).__name__.lower()}")
52         return method(node)
53
54     def _eval_number(self, n): return n.value
55     def _eval_text(self, n): return n.value
56     def _eval_bool(self, n): return n.value
57     def _eval_ref(self, n): return self.value_of(n.ref)
58
59     def _eval_rangenode(self, n):
60         # A bare range outside a function argument is a semantic error.
61         return VALUE_ERR
62
63     def _eval_name(self, n):
64         return NAME_ERR # named ranges arrive in Phase 2
65
66     def _eval_unary(self, n):
67         v = to_number(self.eval(n.operand))
68         return v if is_error(v) else -v
69
70     def _eval_binary(self, n):
71         left = self.eval(n.left)
72         if is_error(left):
73             return left
74         right = self.eval(n.right)
75         if is_error(right):
76             return right
77         op = n.op
78         if op == "&":
79             lt, rt = to_text(left), to_text(right)
80             return lt if is_error(lt) else rt if is_error(rt) else lt + rt
81         if op in ("=", "<"):
82             eq = left == right
83             return eq if op == "=" else not eq
84         if op in ("<=", ">", ">="):
85             if isinstance(left, str) and isinstance(right, str):
86                 a, b = left, right
87             else:
88                 nums = _num_args([left, right])
89                 if is_error(nums):
90                     return nums
91                 a, b = nums
92             return {"<": a < b, "<=": a <= b, ">": a > b, ">=": a >= b}[op]
93         nums = _num_args([left, right])
94         if is_error(nums):
95             return nums
96         a, b = nums
97         if op == "+": return a + b
98         if op == "-": return a - b
99         if op == "*": return a * b
100        if op == "/": return DIV0 if b == 0 else a / b
101        if op == "%": return DIV0 if b == 0 else a % b

```

```

102         if op == "^": return a ** b
103         return VALUE_ERR
104
105     # -- functions -----
106     def _flatten(self, arg_nodes):
107         """Evaluate args, expanding ranges to their non-blank cell values."""
108         vals = []
109         for node in arg_nodes:
110             if isinstance(node, P.RangeNode):
111                 for ref in node.rng.cells():
112                     v = self.value_of(ref)
113                     if v is not BLANK:
114                         vals.append(v)
115             else:
116                 vals.append(self.eval(node))
117         return vals
118
119     def _eval_call(self, n):
120         fname = n.fname
121         if fname == "IF":
122             if len(n.args) != 3:
123                 return VALUE_ERR
124             cond = self.eval(n.args[0])
125             if is_error(cond):
126                 return cond
127             return self.eval(n.args[1] if cond else n.args[2]) # lazy branch
128         if fname in ("SUM", "AVG", "MIN", "MAX", "COUNT"):
129             vals = self._flatten(n.args)
130             if fname == "COUNT": # counts numeric entries only
131                 numeric = [v for v in vals
132                           if isinstance(v, (int, float)) and not isinstance(v, bool)]
133                 return float(len(numeric))
134             nums = _num_args(vals)
135             if is_error(nums):
136                 return nums
137             if not nums:
138                 return DIV0 if fname == "AVG" else 0.0
139             if fname == "SUM": return float(sum(nums))
140             if fname == "AVG": return sum(nums) / len(nums)
141             if fname == "MIN": return float(min(nums))
142             if fname == "MAX": return float(max(nums))
143         if fname == "ABS":
144             v = to_number(self.eval(n.args[0])) if len(n.args) == 1 else VALUE_ERR
145             return v if is_error(v) else abs(v)
146         if fname == "ROUND":
147             if len(n.args) not in (1, 2):
148                 return VALUE_ERR
149             v = to_number(self.eval(n.args[0]))
150             d = to_number(self.eval(n.args[1])) if len(n.args) == 2 else 0
151             if is_error(v): return v
152             if is_error(d): return d
153             return float(round(v, int(d)))
154         if fname == "LEN":
155             t = to_text(self.eval(n.args[0])) if len(n.args) == 1 else VALUE_ERR
156             return t if is_error(t) else float(len(t))
157         return ErrorValue("#NAME?", f"unknown function {fname}")

```

A.6 `tabula/engine.py` — Grid, edit controller, diagnostics, explain (108 lines)

```

1  """Phase 1 grid + edit controller: set/get/show with full recomputation."""
2  from __future__ import annotations
3
4  from dataclasses import dataclass
5
6  from .evaluator import Evaluator
7  from .lexer import LexError, tokenize
8  from .parser import ParseError, dump_ast, parse_formula
9  from .refs import CellRef, index_to_col
10 from .values import BLANK, display
11
12

```

```

13 @dataclass
14 class Cell:
15     raw: str
16     kind: str          # 'literal' | 'formula' | 'broken'
17     payload: object    # value for literal, AST for formula, None for broken
18
19
20 def caret_diagnostic(cellname: str, category: str, message: str,
21                      formula: str, pos: int) -> str:
22     # pos is within the formula body; +1 accounts for the leading '='
23     return (
24         f"error[{category}] in {cellname}: {message}\n"
25         f"  = {formula}\n"
26         f"  {' ' * (pos + 1)}^"
27     )
28
29
30 class Sheet:
31     def __init__(self):
32         self.cells: dict[tuple[int, int], Cell] = {}
33         self.evaluator = Evaluator(self)
34
35     # -- editing -----
36     def set(self, refname: str, raw: str) -> str | None:
37         """Set a cell. Returns a diagnostic string on compile error, else None."""
38         ref = CellRef.parse(refname)
39         if ref is None:
40             return f"error[RefError]: invalid cell reference '{refname}'"
41         raw = raw.strip()
42         if raw.startswith("="):
43             body = raw[1:]
44             try:
45                 ast = parse_formula(body)
46                 self.cells[ref.key()] = Cell(raw, "formula", ast)
47                 return None
48             except LexError as e:
49                 self.cells[ref.key()] = Cell(raw, "broken", None)
50                 return caret_diagnostic(str(ref), "LexicalError",
51                                         e.message, body, e.pos)
52             except ParseError as e:
53                 self.cells[ref.key()] = Cell(raw, "broken", None)
54                 return caret_diagnostic(str(ref), "SyntaxError", e.message, body, e.pos)
55         self.cells[ref.key()] = Cell(raw, "literal", _parse_literal(raw))
56         return None
57
58     def value(self, refname: str):
59         ref = CellRef.parse(refname)
60         return self.evaluator.value_of(ref) if ref else None
61
62     # -- rendering -----
63     def show(self) -> str:
64         if not self.cells:
65             return "(empty sheet)"
66         cols = sorted({c for c, _ in self.cells})
67         rows = sorted({r for _, r in self.cells})
68         widths = {}
69         table = {}
70         for c in cols:
71             col_cells = [display(self.evaluator.value_of(CellRef(c, r))) for r in rows]
72             for r, text in zip(rows, col_cells):
73                 table[(c, r)] = text
74             widths[c] = max([len(index_to_col(c))] + [len(t) for t in col_cells]) + 2
75         head = " " + "".join(index_to_col(c).ljust(widths[c]) for c in cols)
76         lines = [head]
77         for r in rows:
78             row = f"{r:<5}" + "".join(table[(c, r)].ljust(widths[c]) for c in cols)
79             lines.append(row.rstrip())
80         return "\n".join(lines)
81
82     def explain(self, refname: str) -> str:
83         ref = CellRef.parse(refname)
84         cell = self.cells.get(ref.key()) if ref else None
85         if cell is None:
86             return f"{refname}: (blank)"

```

```

87         out = [f"cell    : {ref}", f"row    : {cell.raw}", f"kind    : {cell.kind}"]
88         if cell.kind == "formula":
89             toks = tokenize(cell.raw[1:])
90             out.append("tokens : " + " ".join(
91                 f"{t.lexeme}" for t in toks if t.lexeme))
92             out.append("types  : " + " ".join(
93                 t.type.name for t in toks if t.lexeme))
94             out.append("ast    :")
95             out.extend(" " + line for line in dump_ast(cell.payload))
96         out.append(f"value    : {display(self.evaluator.value_of(ref))}")
97         return "\n".join(out)
98
99
100 def _parse_literal(raw: str):
101     if raw == "":
102         return BLANK
103     if raw.upper() in ("TRUE", "FALSE"):
104         return raw.upper() == "TRUE"
105     try:
106         return float(raw)
107     except ValueError:
108         return raw # text literal

```

A.7 `tabula/cli.py` — REPL: interactive and batch modes (77 lines)

```

1  """Tabula Phase 1 REPL.
2
3  Usage: python -m tabula.cli          interactive
4         python -m tabula.cli script.txt batch (echoes commands, for demos)
5  """
6  from __future__ import annotations
7
8  import sys
9
10 from .engine import Sheet
11
12 HELP = """commands:
13   set <cell> <literal | =formula>   edit a cell
14   get <cell>                        value + raw content
15   show                               render the grid
16   explain <cell>                    tokens, AST, value for a cell
17   help | quit"""
18
19
20 def run_command(sheet: Sheet, line: str) -> str | None:
21     parts = line.strip().split(None, 2)
22     if not parts:
23         return None
24     cmd = parts[0].lower()
25     if cmd == "quit":
26         return "QUIT"
27     if cmd == "help":
28         return HELP
29     if cmd == "set" and len(parts) == 3:
30         return sheet.set(parts[1], parts[2])
31     if cmd == "get" and len(parts) >= 2:
32         ref = parts[1]
33         cell = None
34         from .refs import CellRef
35         r = CellRef.parse(ref)
36         if r:
37             cell = sheet.cells.get(r.key())
38         from .values import display
39         raw = f"({cell.raw})" if cell and cell.kind != "literal" else ""
40         return f"{ref.upper()} = {display(sheet.value(ref))}{raw}"
41     if cmd == "show":
42         return sheet.show()
43     if cmd == "explain" and len(parts) >= 2:
44         return sheet.explain(parts[1])
45     return f"unrecognized command: {line.strip()!r} (try 'help')"
46

```

```

47
48 def main() -> None:
49     sheet = Sheet()
50     if len(sys.argv) > 1: # batch mode: echo each command, then its output
51         with open(sys.argv[1]) as f:
52             for line in f:
53                 line = line.rstrip("\n")
54                 if not line.strip() or line.lstrip().startswith("#"):
55                     continue
56                 print(f"tabula> {line}")
57                 out = run_command(sheet, line)
58                 if out == "QUIT":
59                     break
60                 if out:
61                     print(out)
62     return
63     print("Tabula prototype - type 'help' for commands")
64     while True:
65         try:
66             line = input("tabula> ")
67         except EOFError:
68             break
69         out = run_command(sheet, line)
70         if out == "QUIT":
71             break
72         if out:
73             print(out)
74
75
76 if __name__ == "__main__":
77     main()

```

A.8 tests/test_lexer.py — Scanner tests (60 lines)

```

1  """Unit tests for the formula scanner (task T1.6)."""
2  import pytest
3
4  from tabula.lexer import LexError, TokenType, tokenize
5
6
7  def types(text):
8      return [t.type for t in tokenize(text) if t.type is not TokenType.EOF]
9
10
11 def test_every_token_class():
12     toks = tokenize('SUM(A1:$B$2) + 3.5 & "hi" <> TRUE')
13     assert [t.type for t in toks] == [
14         TokenType.IDENT, TokenType.LPAREN, TokenType.CELLREF, TokenType.COLON,
15         TokenType.CELLREF, TokenType.RPAREN, TokenType.PLUS, TokenType.NUMBER,
16         TokenType.AMP, TokenType.STRING, TokenType.NE, TokenType.BOOL, TokenType.EOF,
17     ]
18
19
20 @pytest.mark.parametrize("text,value", [("12", 12.0), ("3.5", 3.5), (".5", 0.5)])
21 def test_number_forms(text, value):
22     assert tokenize(text)[0].value == value
23
24
25 def test_string_escape_is_doubled_quote():
26     assert tokenize('"say ""hi""')[0].value == 'say "hi"'
27
28
29 def test_absolute_reference_keeps_anchors():
30     ref = tokenize("$B$2")[0].value
31     assert (ref.col, ref.row, ref.abs_col, ref.abs_row) == (2, 2, True, True)
32
33
34 def test_two_char_operators_win_over_one_char():
35     assert types("a<=b") == [TokenType.IDENT, TokenType.LE, TokenType.IDENT]
36     assert types("a<>b") == [TokenType.IDENT, TokenType.NE, TokenType.IDENT]
37     assert types("a<b") == [TokenType.IDENT, TokenType.LT, TokenType.IDENT]

```

```

38
39
40 def test_identifier_vs_reference_classification():
41     assert types("A1") == [TokType.CELLREF]
42     assert types("A1X") == [TokType.IDENT] # not a reference shape
43     assert types("ZZ10000") == [TokType.CELLREF]
44     assert types("ZZ10001") == [TokType.IDENT] # out of bounds -> not a ref
45
46
47 def test_positions_are_recorded():
48     toks = tokenize("1 + 23")
49     assert [t.pos for t in toks[:3]] == [0, 2, 4]
50
51
52 @pytest.mark.parametrize("bad,message", [
53     ('unterminated', "unterminated string literal"),
54     ("1 # 2", "illegal character"),
55     ("$X", "malformed cell reference"),
56 ])
57 def test_lexical_errors(bad, message):
58     with pytest.raises(LexError) as exc:
59         tokenize(bad)
60     assert message in exc.value.message

```

A.9 tests/test_parser.py — Parser tests: precedence, associativity, error positions (64 lines)

```

1  """Parser tests: AST shape, precedence, associativity (task T1.7)."""
2  import pytest
3
4  from tabula.parser import (Binary, Bool, Call, Number, ParseError, RangeNode,
5                             Ref, Text, Unary, parse_formula)
6
7
8  def shape(node):
9      """Fully parenthesized rendering, so precedence is directly assertable."""
10     if isinstance(node, Number):
11         whole = node.value == int(node.value)
12         return str(int(node.value)) if whole else str(node.value)
13     if isinstance(node, Text):
14         return f'"{node.value}"'
15     if isinstance(node, Bool):
16         return "TRUE" if node.value else "FALSE"
17     if isinstance(node, Ref):
18         return str(node.ref)
19     if isinstance(node, RangeNode):
20         return str(node.rng)
21     if isinstance(node, Unary):
22         return f"({shape(node.operand)})"
23     if isinstance(node, Binary):
24         return f"({shape(node.left)} {node.op} {shape(node.right)})"
25     if isinstance(node, Call):
26         return f"{node.fname}({','.join(shape(a) for a in node.args)})"
27     return repr(node)
28
29
30 @pytest.mark.parametrize("src,expected", [
31     ("2+3*4", "(2 + (3 * 4))"), # * binds tighter than +
32     ("2*3+4", "((2 * 3) + 4)"),
33     ("1+2-3", "((1 + 2) - 3)"), # left-associative
34     ("2^3^2", "(2 ^ (3 ^ 2))"), # ^ is RIGHT-associative
35     ("~A1^2", "((-A1) ^ 2)"), # unary binds tighter than ^
36     ("a&1+2", "(a & (1 + 2))"), # & is looser than +
37     ("1+2<3*4", "((1 + 2) < (3 * 4))"), # comparison is loosest
38     ("(2+3)*4", "((2 + 3) * 4)"),
39 ])
40 def test_precedence_and_associativity(src, expected):
41     assert shape(parse_formula(src)) == expected
42
43
44 def test_range_and_call():
45     assert shape(parse_formula("SUM(A1:B4)")) == "SUM(A1:B4)"

```

```

46     assert shape(parse_formula("IF(A1>0,1,2)")) == "IF((A1 > 0), 1, 2)"
47
48
49 def test_nested_calls():
50     assert shape(parse_formula("MAX(SUM(A1:A3),AVG(B1:B3))")) == \
51         "MAX(SUM(A1:A3), AVG(B1:B3))"
52
53
54 @pytest.mark.parametrize("bad,message,pos", [
55     ("SUM(A1:A3", "expected ')' after argument list", 9),
56     ("2+", "unexpected end of formula", 2),
57     ("(1+2", "expected ')'", 4),
58     ("1 2", "unexpected", 2),
59 ])
60 def test_syntax_errors_report_message_and_position(bad, message, pos):
61     with pytest.raises(ParseError) as exc:
62         parse_formula(bad)
63     assert message in exc.value.message
64     assert exc.value.pos == pos

```

A.10 tests/test_evaluator.py — Evaluation, coercion, error-value and cycle tests (96 lines)

```

1  """Evaluation and error-value tests over the prototype grid (task T1.8)."""
2  import pytest
3
4  from tabula.engine import Sheet
5  from tabula.values import display
6
7
8  def sheet_with(**cells) -> Sheet:
9      s = Sheet()
10     for name, raw in cells.items():
11         s.set(name, raw)
12     return s
13
14
15 @pytest.mark.parametrize("formula,expected", [
16     ("=2+3*4", "14"),
17     ("=2^3^2", "512"), # right-associative
18     ("=(2+3)*4", "20"),
19     ("=-3^2", "9"), # (-3)^2 per unary-binds-tighter rule
20     ("=7/2", "3.5"),
21     ("=7%3", "1"),
22     ("='a'&'b'", "ab"),
23     ("='n'&'5", "n=5"), # number coerced to text by &
24     ("=2<3", "TRUE"),
25     ("=2=2", "TRUE"),
26     ("=2<>2", "FALSE"),
27 ])
28 def test_expression_semantics(formula, expected):
29     s = sheet_with(A1=formula)
30     assert display(s.value("A1")) == expected
31
32
33 def test_references_and_aggregates():
34     s = sheet_with(A1="10", A2="25", A3="40",
35                   B1="=SUM(A1:A3)", B2="=AVG(A1:A3)",
36                   B3="=MIN(A1:A3)", B4="=MAX(A1:A3)", B5="=COUNT(A1:A3)")
37     assert display(s.value("B1")) == "75"
38     assert display(s.value("B2")) == "25"
39     assert display(s.value("B3")) == "10"
40     assert display(s.value("B4")) == "40"
41     assert display(s.value("B5")) == "3"
42
43
44 def test_edit_propagates_to_dependents():
45     s = sheet_with(A1="10", A2="25", B1="=SUM(A1:A2)*2")
46     assert display(s.value("B1")) == "70"
47     s.set("A2", "35")
48     assert display(s.value("B1")) == "90"
49

```

```

50
51 def test_blank_cell_reads_as_zero_and_empty_text():
52     s = sheet_with(A1="=B9+5", A2='=B9&"x"')
53     assert display(s.value("A1")) == "5"
54     assert display(s.value("A2")) == "x"
55
56
57 def test_if_evaluates_only_the_taken_branch():
58     # If IF were eager, the untaken 1/0 branch would surface #DIV/0!.
59     s = sheet_with(A1="=IF(TRUE, 42, 1/0)")
60     assert display(s.value("A1")) == "42"
61
62
63 @pytest.mark.parametrize("formula,code", [
64     ("=1/0", "#DIV/0!"),
65     ("=5%0", "#DIV/0!"),
66     ("='abc'*2", "#VALUE!"), # text is never implicitly numeric
67     ("=NOSUCH(1)", "#NAME?"),
68 ])
69 def test_error_values(formula, code):
70     s = sheet_with(A1=formula)
71     assert display(s.value("A1")) == code
72
73
74 def test_errors_propagate_through_dependents():
75     s = sheet_with(A1="=1/0", B1="=A1+1")
76     assert display(s.value("B1")) == "#DIV/0!"
77
78
79 def test_circular_reference_is_detected():
80     s = sheet_with(A1="=B1", B1="=A1")
81     assert display(s.value("A1")) == "#CYCLE!"
82     assert display(s.value("B1")) == "#CYCLE!"
83
84
85 def test_broken_formula_is_contained_to_its_cell():
86     s = sheet_with(A1="I0", B1="=SUM(A1", C1="=A1*2")
87     assert display(s.value("B1")) == "#VALUE!" # broken cell
88     assert display(s.value("C1")) == "20" # neighbours unaffected
89
90
91 def test_compile_error_reports_caret_diagnostic():
92     s = Sheet()
93     diag = s.set("C1", "=SUM(A1:A3")
94     assert "error[SyntaxError] in C1" in diag
95     assert "expected ')' after argument list" in diag
96     assert diag.splitlines()[-1].strip() == "^"

```

A.11 examples/demo.txt — Review 1 demonstration script (27 lines)

```

1 # Tabula Phase 1 demonstration script
2 # Run: python3 -m tabula.cli examples/demo.txt
3
4 # --- 1. literals and formulas over a range
5 set A1 10
6 set A2 25
7 set A3 40
8 set B1 =SUM(A1:A3)*2
9 set B2 =AVG(A1:A3)
10 set B3 =IF(B2>20, "high", "low")
11 show
12
13 # --- 2. editing an input updates dependent formulas
14 set A2 35
15 get B1
16
17 # --- 3. compiler pipeline for one cell
18 explain B1
19
20 # --- 4. error handling: syntax error is reported and contained
21 set C1 =SUM(A1:A3

```

```
22
23 # --- 5. runtime error value and circular reference
24 set C2 =10/0
25 set A5 =B5
26 set B5 =A5
27 show
```

Total: 1041 lines across 11 files.